

CS3211 Technical Reference

ngmh

04/05/2026

1 C++ Threads

```
1 // function
2 void hello() {
3     std::cout << "Hello";
4 }
5
6 // function object
7 class task {
8 public:
9     void operator()() const {
10         hello();
11     }
12 }
13
14 int main() {
15     std::thread t(hello);
16     t.join();
17
18     task f;
19     std::thread my_thread(f);
20 }
```

1.1 Most Vexing Parse

The following code defines a function that:

- Takes in a pointer to a function with no parameters and returns a task
- Returns a thread object

```
1 std::thread my_thread(task());
```

Instead, to create a thread, it should be:

```
1 std::thread my_thread((task()));
2 std::thread my_thread{task()};
3 std::thread my_thread([]{ hello(); });
```

1.2 Waiting and Detaching

Always join even when an exception occurs! If you forget to join a joinable thread, `std::terminate` is called.

```
1 t.join();
2 t.joinable(); // true
3
4 t.detach();
5 t.joinable(); // false
```

1.3 Passing Arguments to Threads

Pass in references if necessary.

```
1 void f(int i);
2 std::thread t(f, 3);
3
4 void update(widget w, data& data);
5 std::thread t(update, w, std::ref(data));
6 // necessary! constructor copies / moves objects
```

1.4 Thread Ownership

```
1 // out of function
2 std::thread f() {
3     void foo();
4     return std::thread{foo};
5 }
6
7 // into function
8 void f(std::thread t)
9 void g() {
10     void func();
11     f(std::thread{func});
12     // or
13     std::thread t{func};
14     f(std::move(t));
15 }
```

2 C++ Synchronisation

2.1 RAII

- Resource Acquisition Is Initialisation
- Binds life cycle of resource that must be acquired before use to the lifetime of an object
- Resource is released when lifetime of controlling object ends in reverse order of acquisition

2.2 Using Mutexes

```
1 std::mutex mut;
2 void f() {
3     std::lock_guard<std::mutex> guard{mut};
4     // critical section
5 }
```

2.3 Types of Locks

```
1 std::mutex m1, m2;
2
3 void f() {
4     // lock_guard is the simplest
5     // can transfer ownership of lock
6     std::lock(m1, m2);
7     std::lock_guard<std::mutex> lock1(m1, std::adopt_lock);
8     std::lock_guard<std::mutex> lock2(m2, std::adopt_lock);
9
10    // unique lock ownership can be transferred
11    // can manually lock and unlock with unique lock
12    std::unique_lock<std::mutex> u_lock{m1, std::defer_lock};
13    u_lock.lock();
14    u_lock.unlock();
15    // or
16    std::lock(u_lock);
17
18    // scoped lock can take in multiple mutexes, ensures no deadlock
19    std::scoped_lock<std::mutex, std::mutex> guard{m1, m2};
20 }
```

2.4 Condition Variables

```
1 std::mutex mut;
2 std::queue<int> q;
3 std::condition_variable data_cond;
4
5 void producer() {
6     {
7         std::lock_guard<std::mutex> lk{mut};
8         q.push(1);
9     }
10    data_cond.notify_one();
11 }
12
13 void consumer() {
14     while (true) {
```

```

15     // condition_variable needs unique_lock
16     // condition_variable_any does not
17     std::unique_lock<std::mutex> lk{mut};
18     data_cond.wait(lk, []{ return !q.empty(); });
19     // data_cond.wait_for(...)
20     // data_cond.wait_until(...)
21     q.pop();
22 }
23 }

```

3 C++ Memory Orders

3.1 std::memory_order Enumeration

```

1 // Default
2 std::memory_order_seq_cst
3 // Store Operations
4 std::memory_order_seq_cst
5 std::memory_order_release
6 std::memory_order_relaxed
7 // Load Operations
8 std::memory_order_seq_cst
9 std::memory_order_acquire
10 std::memory_order_consume
11 std::memory_order_relaxed
12 // RMW Operations
13 std::memory_order_seq_cst
14 std::memory_order_acq_rel
15 std::memory_order_acquire
16 std::memory_order_release
17 std::memory_order_consume
18 std::memory_order_relaxed

```

3.2 Memory Orderings

```

1 // Sequentially Consistent Ordering
2 std::memory_order_seq_cst
3
4 // Relaxed Ordering
5 std::memory_order_relaxed
6
7 // Acquire-Release Ordering
8 std::memory_order_acquire
9 std::memory_order_release
10 std::memory_order_acq_rel
11 std::memory_order_consume

```

4 Concurrent Data Structures

4.1 S1: Thread Safe Stack with one global mutex

- Safe for multiple threads to call member functions concurrently
- Exception safe
- Work is serialised; only one thread can ever do work, limiting performance
- No means of waiting for item to be added

```
1 struct empty_stack: std::exception {
2     const char* what() const throw();
3 };
4 template<typename T>
5 class threadsafe_stack {
6 private:
7     std::stack<T> data;
8     mutable std::mutex m;
9 public:
10    threadsafe_stack(){}
11    threadsafe_stack(const threadsafe_stack& other) {
12        std::lock_guard<std::mutex> lock(other.m);
13        data = other.data;
14    }
15    threadsafe_stack& operator=(const threadsafe_stack&) = delete;
16    void push(T new_value) {
17        std::lock_guard<std::mutex> lock(m);
18        data.push(std::move(new_value));
19    }
20    std::shared_ptr<T> pop() {
21        std::lock_guard<std::mutex> lock(m);
22        if (data.empty()) throw empty_stack();
23        std::shared_ptr<T> const res(
24            std::make_shared<T>(std::move(data.top())))
25        );
26        data.pop();
27        return res;
28    }
29    void pop(T& value) {
30        std::lock_guard<std::mutex> lock(m);
31        if (data.empty()) throw empty_stack();
32        value = std::move(data.top());
33        data.pop();
34    }
35    bool empty() const {
36        std::lock_guard<std::mutex> lock(m);
37        return data.empty();
38    }
39 };
```

4.2 Q1: Thread Safe Queue with notification

- Analysis from stack also applies
- `wait_and_pop()` provides a solution to waiting for an entry
- If a thread throws an exception after it is woken up, no other threads will be woken
- Can replace with a global notification, or notify one when an exception is thrown, or move shared pointer initialisation to push and store shared pointer instances instead

```
1 template<typename T>
2 class threadsafe_queue {
3 private:
4     mutable std::mutex mut;
5     std::queue<T> data_queue;
6     std::condition_variable data_cond;
7 public:
8     threadsafe_queue() {}
9     void push(T new_value) {
10         std::lock_guard<std::mutex> lk(mut);
11         data_queue.push(std::move(new_value));
12         data_cond.notify_one();
13     }
14     void wait_and_pop(T& value) {
15         std::unique_lock<std::mutex> lk(mut);
16         data_cond.wait(lk, [this]{ return !data_queue.empty(); });
17         value = std::move(data_queue.front());
18         data_queue.pop();
19     }
20     std::shared_ptr<T> wait_and_pop() {
21         std::unique_lock<std::mutex> lk(mut);
22         data_cond.wait(lk, [this]{ return !data_queue.empty(); });
23         std::shared_ptr<T> res(
24             std::make_shared<T>(std::move(data_queue.front())))
25         );
26         data_queue.pop();
27         return res;
28     }
29     bool try_pop(T& value) {
30         std::lock_guard<std::mutex> lk(mut);
31         if (data_queue.empty()) return false;
32         value = std::move(data_queue.front());
33         data_queue.pop();
34         return true;
35     }
36     std::shared_ptr<T> try_pop() {
37         std::lock_guard<std::mutex> lk(mut);
38         if (data_queue.empty()) return std::shared_ptr<T>();
39         std::shared_ptr<T> res(
40             std::make_shared<T>(std::move(data_queue.front())))
```

```

41         );
42         data_queue.pop();
43         return res;
44     }
45     bool empty() const {
46         std::lock_guard<std::mutex> lk(mut);
47         return data_queue.empty();
48     }
49 };

```

4.3 Q2: Push shared pointers in the queue

- Exception safe
- Allocation of new instance can be done outside of the lock for better performance
- Concurrency is still limited by global mutex

```

1  template<typename T>
2  class threadsafe_queue {
3  private:
4      mutable std::mutex mut;
5      std::queue<std::shared_ptr<T>> data_queue;
6      std::condition_variable data_cond;
7  public:
8      threadsafe_queue() {}
9      void wait_and_pop(T& value) {
10         std::unique_lock<std::mutex> lk(mut);
11         data_cond.wait(lk, [this]{ return !data_queue.empty(); });
12         value = std::move(*data_queue.front());
13         data_queue.pop();
14     }
15     bool try_pop(T& value) {
16         std::lock_guard<std::mutex> lk(mut);
17         if (data_queue.empty()) return false;
18         value = std::move(*data_queue.front());
19         data_queue.pop();
20         return true;
21     }
22     std::shared_ptr<T> wait_and_pop() {
23         std::unique_lock<std::mutex> lk(mut);
24         data_cond.wait(lk, [this]{ return !data_queue.empty(); });
25         std::shared_ptr<T> res = data_queue.front();
26         data_queue.pop();
27         return res;
28     }
29     std::shared_ptr<T> try_pop() {
30         std::lock_guard<std::mutex> lk(mut);
31         if (data_queue.empty()) return std::shared_ptr<T>();

```

```

32     std::shared_ptr<T> res = data_queue.front();
33     data_queue.pop();
34     return res;
35 }
36 void push(T new_value) {
37     std::shared_ptr<T> data(
38         std::make_shared<T>(std::move(new_value))
39     );
40     std::lock_guard<std::mutex> lk(mut);
41     data_queue.push(data);
42     data_cond.notify_one();
43 }
44 bool empty() const {
45     std::lock_guard<std::mutex> lk(mut);
46     return data_queue.empty();
47 }
48 };

```

4.4 Q3: Single threaded queue

- Uses unique pointers to manage nodes so that they get deleted by RAI
- Pushing can modify both the front and back
- Pushing and popping access the next pointer of a node which requires serialisation

```

1  template<typename T>
2  class queue {
3  private:
4      struct node {
5          T data;
6          std::unique_ptr<node> next;
7          node(T data_):
8              data(std::move(data_))
9          {}
10 };
11 std::unique_ptr<node> front;
12 node* back;
13 public:
14     queue(): back(nullptr) {}
15     queue(const queue& other)=delete;
16     queue& operator=(const queue& other)=delete;
17     std::shared_ptr<T> try_pop() {
18         if (!front) {
19             return std::shared_ptr<T>();
20         }
21         std::shared_ptr<T> const res(
22             std::make_shared<T>(std::move(front->data))
23         );

```

```

24     std::unique_ptr<node> const old_front = std::move(front);
25     front = std::move(old_front->next);
26     if (!front) back = nullptr;
27     return res;
28 }
29 void push(T new_value) {
30     std::unique_ptr<node> p(new node(std::move(new_value)));
31     node* const new_back = p.get();
32     if (back) back->next = std::move(p);
33     else front = std::move(p);
34     back = new_back;
35 }
36 };

```

4.5 Q4: Single threaded queue, Enabling concurrency by separating data

- Pre-allocate dummy node with no data, so that there is always at least one node in the queue separating the front and back nodes
- Extra level of indirection needed to store data by pointers

```

1  template<typename T>
2  class queue {
3  private:
4      struct node {
5          std::shared_ptr<T> data;
6          std::unique_ptr<node> next;
7      };
8      std::unique_ptr<node> front;
9      node* back;
10 public:
11     queue(): front(new node), back(front.get()) {}
12     queue(const queue& other)=delete;
13     queue& operator=(const queue& other)=delete;
14     std::shared_ptr<T> try_pop() {
15         if (front.get() == back) {
16             return std::shared_ptr<T>();
17         }
18         std::shared_ptr<T> const res(front->data);
19         std::unique_ptr<node> old_front = std::move(front);
20         front = std::move(old_front->next);
21         return res;
22     }
23     void push(T new_value) {
24         std::shared_ptr<T> new_data(
25             std::make_shared<T>(std::move(new_value))
26         );
27         std::unique_ptr<node> p(new node);
28         back->data = new_data;

```

```

29     node* const new_back = p.get();
30     back->next = std::move(p);
31     back = new_back;
32 }
33 };

```

4.6 Q5: Fine-grained Thread-safe Queue

Invariants Held:

- `back->next == nullptr`
- `back->data == nullptr`
- `front == back` implies an empty list
- A single element list has `front->next == back`
- For each node `x` where `x != back`, `x->data` points to an instance of `T` and `x->next` points to the next node in the list; `x->next == back` implies `x` is the last node in the list
- Following the next nodes from `front` will eventually yield `back`

```

1  template<typename T>
2  class threadsafe_queue {
3  private:
4      struct node {
5          std::shared_ptr<T> data;
6          std::unique_ptr<node> next;
7      };
8      std::mutex front_mutex;
9      std::unique_ptr<node> front;
10     std::mutex back_mutex;
11     node* back;
12     node* get_back() {
13         std::lock_guard<std::mutex> back_lock(back_mutex);
14         return back;
15     }
16     std::unique_ptr<node> pop_front() {
17         std::lock_guard<std::mutex> front_lock(front_mutex);
18         if (front.get() == get_back()) {
19             return nullptr;
20         }
21         std::unique_ptr<node> old_front = std::move(front);
22         front = std::move(old_front->next);
23         return old_front;
24     }
25 public:
26     threadsafe_queue(): front(new node), back(front.get()) {}
27     threadsafe_queue(const queue& other)=delete;
28     threadsafe_queue& operator=(const queue& other)=delete;

```

```

29     std::shared_ptr<T> try_pop() {
30         std::unique_ptr<node> old_front = pop_front();
31         return old_front ? old_front->data : std::shared_ptr<T>();
32     }
33     void push(T new_value) {
34         std::shared_ptr<T> new_data(
35             std::make_shared<T>(std::move(new_value))
36         );
37         std::unique_ptr<node> p(new node);
38         node* const new_back = p.get();
39         std::lock_guard<std::mutex> back_lock(back_mutex);
40         back->data = new_data;
41         back->next = std::move(p);
42         back = new_back;
43     }
44 };

```

4.7 Q6: Even more Fine-grained Lock-based Queue

- Per-node mutexes provide synchronises with relationship between push and pop threads
- Additional scope introduced when trying to pop avoids UAF

```

1  class FineQueue {
2      struct Node {
3          std::mutex mut;
4          Node *next = nullptr;
5          Data data{};
6      };
7      std::mutex mut_back;
8      Node *back;
9      std::mutex mut_front;
10     Node *front;
11 public:
12     FineQueue() : mut_back{},
13                 back{new Node{}},
14                 mut_front{},
15                 front{back} {}
16     ~FineQueue() {
17         while (front != nullptr) {
18             Node *next = front->next;
19             delete front;
20             front = next;
21         }
22     }
23     void push(Data data) {
24         Node *new_node = new Node{};
25         std::scoped_lock lock{mut_back};
26         std::scoped_lock node_lock{back->mut};

```

```

27     back->data = data;
28     back->next = new_node;
29     back = new_node;
30 }
31 std::optional<Data> try_pop() {
32     Node *old_node;
33     {
34         std::scoped_lock lock{mut_front};
35         old_node = front;
36         std::scoped_lock node_lock{old_node->mut};
37         if (old_node->next == nullptr) {
38             return std::nullopt;
39         }
40         front = front->next;
41     }
42     Data data = old_node->data;
43     delete old_node;
44     return data;
45 }
46 };

```

5 Testing and Debugging

5.1 Using Valgrind

```

1 valgrind --tool=memcheck <prog>

```

5.2 Using ASan

```

1 clang++ -O1 -fsanitize=address a.cc && ./a.out

```

5.3 How ASan works

```

1 // X is good byte, 0 is bad byte, mapped to shadow value
2 XXXXXXXX 0
3 XXXXXXX0 7
4 XXXXXX00 6
5 XXXXX000 5
6 XXXX0000 4
7 XXX00000 3
8 XX000000 2
9 X0000000 1
10 00000000 -1
11
12 *a=...;
13 char *shadow = addr >> 3; // find corresponding byte

```

```

14 if (*shadow // all safe has value zero, fails this check
15 && *shadow <= ((a&7)+N-1)) { // find offset within block and last
    accessed byte
16     ReportError(a);
17 }
18 *a=...;

```

5.4 Using TSan

```

1 g++ a.cpp -fsanitize=thread -fPIE -pie
2 TSAN_OPTIONS="history-size=7 force_seq_cst_atomics=1" ./myprogram

```

5.5 How TSan works

```

1 Shadow = 4 * (Addr & kMask); // shadow address hash function
2
3 TID: 16 bits (TID, thread ID)
4 Epoc: 42 bits (Epoch, scalar clock)
5 Pos: 5 bits (Position in 8-byte word)
6 IsW: 1 bit (IsWrite)

```

6 Midterms Tutorial Stuff

6.1 C++ Concurrent Queue with Semaphore

```

1 struct Job {
2     int id;
3     int count;
4 };
5
6 class JobQueue {
7     std::queue<Job> jobs;
8     std::mutex mut;
9     std::counting_semaphore<> count;
10
11 public:
12     JobQueue() : jobs{}, mut{}, count{0} {}
13
14     void enqueue(Job job) {
15         std::unique_lock lock{mut};
16         jobs.push(job);
17         count.release(); // increment semaphore counter
18     }
19
20     std::optional<Job> try_dequeue() {
21         if (count.try_acquire()) {

```

```

22     // enter critical section and remove a job
23     std::unique_lock lock{mut};
24     Job job = jobs.front();
25     jobs.pop();
26     return job;
27 } else {
28     // queue is empty
29     return std::nullopt;
30 }
31 }
32
33 Job dequeue() {
34     count.acquire(); // decrement semaphore counter
35     std::unique_lock lock{mut};
36     Job job = jobs.front();
37     jobs.pop();
38     return job;
39 }
40 };

```

6.2 C++ Concurrent Queue with Monitor

```

1 struct Job {
2     int id;
3     int count;
4 }
5 class JobQueue {
6     std::queue<Job> jobs;
7     std::mutex mut;
8     std::condition_variable cond;
9 public:
10     JobQueue() : jobs{}, mut{}, cond{} {}
11     void enqueue(Job job) {
12         {
13             std::unique_lock lock{mut};
14             jobs.push(job);
15         }
16         cond.notify_one();
17     }
18     std::optional<Job> try_dequeue() {
19         std::unique_lock lock{mut};
20         if (jobs.empty()) {
21             return std::nullopt;
22         } else {
23             Job job = jobs.front(); jobs.pop();
24             return job;
25         }
26     }

```

```

27 Job dequeue() {
28     std::unique_lock lock{mut};
29     cond.wait(lock, [this]() { return !jobs.empty(); });
30     // same as
31     // while (jobs.empty()) {
32     //     cond.wait(lock);
33     // }
34     Job job = jobs.front();
35     jobs.pop();
36     return job;
37 }
38 };

```

6.3 Atomic Puzzle Tips

- Find possible modification orders; they must obey write-write coherence.
- Decide the possible outcomes of reads; they must obey read-write, write-read, write-write coherence.
- Deduce the sw relationships between threads; the load must successfully read the store (or value in release sequence) and the two operations must be suitably tagged.
- Deduce the hb relationships.
- For non-atomic variables, any two operations involving at least one write must be synchronised; otherwise, there is a data race.

6.4 Smart Pointers

- Unique pointer implements RAII with unique ownership.
- `std::shared_ptr` stores `ref_count` (thread safe) and the underlying object (not thread safe), protecting shared resources.
- Weak pointer acts as a viewer and helps to prevent circular dependencies since it does not contribute to ownership.
- The underlying object is deleted when `shared_count` hits 0.
- The control block is deleted when `weak_count` hits 0.
- If `shared_count` is above 0, it counts towards `weak_count` as well.

```

1 std::unique_ptr<int> foo = std::make_unique<int>(0);
2 foo.get(); // *foo
3
4 std::shared_ptr<int> x = std::make_shared<int>(100);
5 {
6     auto a = x;
7     auto b = x;
8     cout << x.use_count();
9 }

```

```

10
11 // Shared Pointer Usage
12
13 // Stack (Local Variables)
14 shared_ptr { ctrl_blk* }
15 weak_ptr   { ctrl_blk* }
16
17 // Heap (The Control Block)
18 ctrl_blk {
19     std::atomic<size_t> shared_count; // Controls object life
20     std::atomic<size_t> weak_count;   // Controls control block life
21     T* ptr_to_obj;                   // The actual data
22 }

```

6.5 C++ Shared Pointer

```

1 template <typename T>
2 struct SharedPtr {
3 private:
4     std::atomic<size_t>* m_count;
5     T* m_ptr;
6 public:
7     SharedPtr(T* ptr) : m_count(new std::atomic<size_t>(1)), m_ptr(
8         ptr) { }
9     SharedPtr(const SharedPtr& other) : m_count(other.m_count),
10        m_ptr(other.m_ptr) {
11        m_count->fetch_add(1, std::memory_order::relaxed);
12    }
13    ~SharedPtr() {
14        size_t old_count = m_count->fetch_sub(1, std::memory_order::
15            acq_rel);
16        if(old_count == 1) {
17            delete m_ptr;
18            delete m_count;
19        }
20    }
21    T* get() { return m_ptr; }
22    const T* get() const { return m_ptr; }
23 };

```

6.6 Compare and Swap

```

1 int v = x.load();
2 while (true) {
3     // could spuriously fail
4     if (x.compare_and_exchange_weak(v, v+1)) {
5         return v;
6     }
7 }

```

```

6     }
7 }
8 // or
9 x.compare_and_exchange_strong(v, v+1);
10 // similar to
11 x.fetch_add(1);

```

6.7 C++ Lock Free Stack

```

1 void push(Job job) {
2     Node* newTop = new Node{job, nullptr};
3     Node* oldTop = top.load();
4     while (true) {
5         newTop->next.store(oldTop);
6         if (top.compare_exchange_weak(oldTop, newTop)) {
7             break;
8         }
9     }
10 }
11 std::optional<Job> try_pop() {
12     Node* oldTop = top.load();
13     while (true) {
14         if (oldTop == nullptr) return std::nullopt;
15         Node* newTop = oldTop->next.load();
16         if (top.compare_exchange_weak(oldTop, newTop)) {
17             Job job = oldTop->job;
18             delete oldTop;
19             return job;
20         }
21     }
22 }

```

6.8 C++ Lock Free Queue + Recycling Centre

- Unbounded lock free queue with push and non-blocking try_pop
- Avoids ABA problem using generation counter
- Avoids UAF by reusing nodes instead of deallocating them
- However, does not provide serialisation

```

1 struct Job
2 {
3     int id;
4     int data;
5 };
6 class JobQueue {
7     using stdmo = std::memory_order;

```

```

8
9 // a node is a dummy node if its next pointer is set to
  QUEUE_END
10 // use the next ptr to establish the synchronizes-with
   relationship
11 // next is in charge of releasing job
12 struct Node {
13     std::atomic<Node*> next = QUEUE_END;
14     Job job;
15 };
16
17 // sentinel values
18 static inline Node* const QUEUE_END = nullptr;
19 static inline Node* const STACK_END = QUEUE_END + 1;
20
21 struct alignas(16) GenNodePtr {
22     Node* node;
23     uintptr_t gen;
24 };
25
26 static_assert(std::atomic<GenNodePtr>::is_always_lock_free);
27
28 alignas(64) std::atomic<Node*> m_queue_back; //
   producer end
29 alignas(64) std::atomic<GenNodePtr> m_queue_front; //
   consumer end
30 alignas(64) std::atomic<GenNodePtr> m_recycled_stack_top; //
   recycled node stack
31
32 public:
33 // queue starts with a dummy node
34 JobQueue()
35     : m_queue_back(new Node())
36     , m_queue_front(GenNodePtr { m_queue_back.load(stdmo::
   relaxed), 1 })
37     , m_recycled_stack_top(GenNodePtr { STACK_END, 1 }) {}
38
39 ~JobQueue() {
40     // assumes no other threads are accessing the job queue
41     Node* cur_node = m_queue_front.load(stdmo::relaxed).node;
42     while(cur_node != QUEUE_END) {
43         Node* next = cur_node->next;
44         delete cur_node;
45         cur_node = next;
46     }
47
48     // clean up the recycled nodes
49     cur_node = m_recycled_stack_top.load(stdmo::relaxed).node;
50     while(cur_node != STACK_END) {

```

```

51         Node* next = cur_node->next;
52         delete cur_node;
53         cur_node = next;
54     }
55 }
56
57 // either get a node from the recycling stack if we have some
58 // or allocate a new one if we dont
59 Node* get_recycled_node_or_allocate_new() {
60     GenNodePtr old_stack_top = m_recycled_stack_top.load(stdmo::
        relaxed);
61     while(true) {
62         if(old_stack_top.node == STACK_END) {
63             return new Node();
64         }
65
66         // use acquire, synchronise with the release-store of
67         // node->next in add_node_to_recycling_stack
68         Node* cur_stack_next = old_stack_top.node->next.load(
            stdmo::acquire);
69
70         GenNodePtr new_stack_top { cur_stack_next, old_stack_top
            .gen + 1 };
71
72         if(m_recycled_stack_top.compare_exchange_weak(
73             old_stack_top,
74             new_stack_top,
75             stdmo::relaxed)) {
76             return old_stack_top.node;
77         }
78     }
79 }
80
81 // put node in recycling centre
82 void add_node_to_recycling_stack(Node* node) {
83     GenNodePtr old_stack_top = m_recycled_stack_top.load(stdmo::
        relaxed);
84     while(true) {
85         // use release, synchronise with the acquire-load of
86         // node->next in get_recycled_node_or_allocate_new
87         node->next.store(old_stack_top.node, stdmo::release);
88
89         GenNodePtr new_stack_top { node, old_stack_top.gen + 1
            };
90
91         if(m_recycled_stack_top.compare_exchange_weak(
92             old_stack_top,
93             new_stack_top,
94             stdmo::relaxed)) {

```

```

95         break;
96     }
97 }
98 }
99 public:
100 void push(Job job) {
101     Node* new_dummy = get_recycled_node_or_allocate_new();
102     new_dummy->next.store(QUEUE_END, stdmo::relaxed);
103
104     Node* work_node = m_queue_back.exchange(new_dummy, stdmo::
        acq_rel);
105     work_node->job = job;
106     work_node->next.store(new_dummy, stdmo::release);
107 }
108
109 std::optional<Job> try_pop() {
110     // splice node from the front of queue, but only if its not
111     dummy
112     // successfully splicing a node establishes global order of
113     pops
114
115     GenNodePtr old_front = m_queue_front.load(stdmo::acquire);
116     while(true) {
117         // get node out from the GenNodePtr
118         Node* old_front_next = old_front->next.load(stdmo::
            acquire);
119         if(old_front_next == QUEUE_END) {
120             return std::nullopt;
121         }
122
123         // note that the generation is strictly increasing
124         GenNodePtr new_front { old_front_next, old_front.gen + 1
            };
125
126         if(m_queue_front.compare_exchange_weak(old_front,
            new_front, stdmo::acq_rel)) {
127             // node now belongs to us
128             break;
129         }
130     }
131
132     Job job = old_front->job;
133     add_node_to_recycling_stack(old_front->node);
134     return job;
135 };

```

7 Go Basics

7.1 Fundamentals

```
1 world := "World"
2 var exclamation string = "!" // name before type
3 defer func(){}() // RAII style, FILO
4
5 if res, err := something(); err != nil {
6
7 }
8
9 s := make([]int, 0, 5) // size 0, cap 5
10 s = append(s, 0) // append 0 to slice
11
12 m1 := make(map[string]int)
13 m2 := make[string]int{"A": 65}
14 val, ok := m1[key1]
15 // iteration order is random
16
17 // set trick
18 s := map[string]struct{}{}
19 _, exist := s["A"]
20
21 for i := 0; i < 5; i++ {}
22 for {
23     if ok := check(); !ok { break }
24 }
25 for check() {}
26 for i, elem := range []int{1, 2, 3} {} // use _ if unwanted
27 for k, v := range map[int]int{} {}
28 for elem := range elemCh {}
29
30 switch num {
31 case 1: smth()
32 case 2: fallthrough
33 default: default()
34 }
35
36 type Duck interface { Walk() }
37 type SomeBird struct { wing Wing }
38 func (b *SomeBird) Walk() {} // tied to an instance, by reference
39 // SomeBird fulfils Duck even if not explicitly linked
```

7.2 Goroutines

```
1 func hello() {
2     fmt.Println("hello")
}
```

```

3 }
4 func main() {
5     go hello()
6     go func() { fmt.Println("bye") }() // anonymous function
7 }

```

7.3 Synchronising Goroutines

Use WaitGroup.

```

1 func main() {
2     var wg sync.WaitGroup
3     for _, x := range []string{"a", "b", "c"} {
4         wg.Add(1)
5         // closure over x
6         go func() {
7             defer wg.Done() // RAII style
8             fmt.Println(x)
9         }
10        // copy x
11        go func(x string) {
12            defer wg.Done()
13            fmt.Println(x)
14        }
15    }
16    wg.Wait()
17 }

```

7.4 Channels

```

1 // bidirectional channels
2 var x chan interface{}
3 x = make(chan interface{})
4
5 // unidirectional channels
6 var rx <-chan interface{}
7 var tx chan<- interface{}
8 x := make(chan interface{})
9 rx = x
10 tx = x
11
12 // buffered channel
13 c := make(chan interface{}, 4)
14
15 // using channels
16 s := make(chan string)
17 go func() {
18     s <- "hi"

```

```

19 }()
20 t, ok := <-s
21 // ok indicates whether read was a write or default value
22 // reading from closed channel is always allowed

```

7.5 Synchronisation with Channels

```

1 // ranging over channel
2 c := make(chan int)
3 go func() {
4     defer close(c)
5     for i := 1; i <= 5; i++ {
6         c <- i
7     }
8 }()
9 for x := range c {
10     fmt.Println(x)
11 }
12
13 c := make(chan interface{})
14 var wg sync.WaitGroup
15 for i := 0; i < 5; i++ {
16     wg.Add(1)
17     go func(i int) {
18         defer wg.Done()
19         <-c
20     }
21 }
22 close(c)
23 wg.Wait()

```

7.6 select

```

1 // pseudorandom uniform selection
2 // no fall through
3 c1 := make(chan interface{}); close(c1)
4 c2 := make(chan interface{}); close(c2)
5 var x1, x2 int
6 for i := 1000; i >= 0; i-- {
7     select {
8         case <- c1: x1++
9         case <- c2: x2++
10        default:
11    }
12 }
13
14 // timeout channel implementation

```

```

15 select {
16     case <-c:
17     case <- time.After(1 * time.Second):
18         fmt.Println("Timed out")
19 }
20
21 // for-select
22 loop:
23 for {
24     select {
25         case <- done: break loop
26         default:
27     }
28     // do work
29 }

```

7.7 Synchronisation Constructs

```

1 // RWMutex
2 var mu sync.RWMutex
3 mu.RLock() // multiple readers can hold
4 defer mu.RUnlock()
5
6 // similar to Mutex
7 mu.Lock() // only one writer can hold
8 defer mu.Unlock()
9
10 // Once
11 var once sync.Once
12 // excutes only once
13 once.Do(func() {
14     // something
15 })
16
17 // Cond
18 mu := &sync.Mutex{}
19 cond := sync.NewCond(mu)
20 ready := false
21 // waiter
22 go func() {
23     mu.Lock()
24     for !ready {
25         cond.Wait()
26     }
27     fmt.Println("Ready!")
28     mu.Unlock()
29 }()
30 // In the signaller

```

```

31 mu.Lock()
32 ready = true
33 cond.Signal() // wake one
34 cond.Broadcast() // wake all
35 mu.Unlock()
36
37 // Pool
38 var bufferPool = sync.Pool{
39     New: func() interface{} {
40         return make([]byte, 1024)
41     },
42 }
43 func process() {
44     buf := bufferPool.Get().([]byte)
45     defer bufferPool.Put(buf)
46     // use buffer
47 }

```

8 Go Patterns

8.1 Lexical Confinement

```

1 // only expose reading end of channel to others
2 owner := func() <-chan int {
3     res := make(chan int, 5)
4     go func() {
5         defer close(res)
6         for i := 0; i <= 5; i++ {
7             res <- i
8         }
9     }()
10    return res
11 }
12 consumer := func(res <-chan int) {
13     for res := range res {
14         fmt.Println(res)
15     }
16 }
17 res := owner()
18 consumer(res)
19
20 printData := func(data []byte) {
21     // ...
22 }
23 // expose only slices of arrays
24 go printData(data[:3])
25 go printData(data[3:])

```

8.2 For-Select Loop

```
1 for {
2     select {
3         case num := <-q:
4             sum += num
5         case <-done:
6             sumCh <- sum
7             return
8     }
9 }
```

8.3 Preventing Leaking

```
1 // returns status channel
2 doWork := func(
3     done <-chan interface{},
4     strings <- chan string,
5 ) <-chan interface{} {
6     terminated := make(chan interface{})
7     go func() {
8         defer close(terminated)
9         for {
10             select {
11                 case s := <-strings: fmt.Println(s)
12                 case <-done: return
13             }
14         }
15     }
16 }
17
18 // done channel
19 done := make(chan interface{})
20 terminated := doWork(done, nil)
21 go func() {
22     time.Sleep(1 * time.Second)
23     close(done)
24 }
25 <-terminated
```

8.4 Error Handling

```
1 type Result struct {
2     Error error
3     Response *http.Response
4 }
5 checkStatus := func(done <-chan interface{}, urls ...string) <-chan
    Result {
```

```

6     results := make(chan Result)
7     go func() {
8         defer close(results)
9         for _, url := range urls {
10             var result Result
11             resp, err := http.Get(url)
12             result = Result{Error: err, Response: resp}
13             select {
14                 case <-done: return
15                 case results <- result:
16             }
17         }
18         return results
19     }
20 }
21 done := make(chan interface{})
22 defer close(done)
23 urls := []string{"", ""}
24 for res := range checkStatus(done, urls...) {
25     if res.Error != nil {
26         continue
27     }
28 }

```

8.5 Pipeline

```

1 generator := func(done <-chan interface{}, integers ...int) <-chan
  int {
2     intStream := make(chan int)
3     go func() {
4         defer close(intStream)
5         for _, i := range integers {
6             select {
7                 case <-done: return
8                 case intStream <- i:
9             }
10        }
11    }()
12    return intStream
13 }
14 multiply := func(
15     done <-chan interface{},
16     intStream <-chan int,
17     multiplier int,
18 ) <-chan int {
19     multipliedStream := make(chan int)
20     go func() {
21         defer close(multipliedStream)

```

```

22     for i := range intStream {
23         select {
24             case <-done: return
25             case multipliedStream <- i * multiplier:
26                 // add is the same but with + here
27         }
28     }
29 }()
30 return multipliedStream
31 }
32
33 done := make(chan interface{})
34 defer close(done)
35 intStream := generator(done, 1, 2, 3, 4)
36 // * 2 + 1 * 2
37 pipeline := multiply(done, add(done, multiply(done, intStream, 2),
38     1), 2)
39 for v := range pipeline {
40     fmt.Println(v)
41 }

```

8.6 Fan-out, Fan-in

```

1 fanIn := func(
2     done <-chan interface{},
3     channels ...<-chan interface{}
4 ) <-chan interface {
5     var wg sync.WaitGroup
6     multiplexedStream := make(chan interface{})
7     multiplex := func(c <-chan interface{}) {
8         defer wg.Done()
9         for i := range c {
10             select {
11                 case <-done: return
12                 case multiplexedStream <- i:
13             }
14         }
15     }
16     wg.Add(len(channels))
17     for _, c := range channels {
18         go multiplex(c)
19     }
20     go func() {
21         wg.Wait()
22         close(multiplexedStream)
23     }()
24     return multiplexedStream
25 }

```

```

26
27 done := make(chan interface{})
28 defer close(done)
29 start := time.Now()
30 rand := func() interface{} { return rand.Intn(676767) }
31 randIntStream := toInt(done, repeatFn(done, rand))
32 numFinders := runtime.NumCPU() // number of OS threads
33 finders := make([]chan interface{}, numFinders)
34 for i := 0; i < numFinders; i++ {
35     finders[i] = primeFinder(done, randIntStream)
36 }
37 // limit output
38 for prime := range take(done, fanIn(done, finders...), 10) {
39     fmt.Println(prime)
40 }

```

8.7 Load Balancer

```

1 // requests sent to balancer
2 type Request struct {
3     fn func() int // operation to perform
4     c chan int // channel for result
5 }
6
7 // requester
8 func requester(work chan<- Request) {
9     c := make(chan int)
10    for {
11        work <- Request{ workFn, c }
12        result := <-c
13        process(result)
14    }
15 }
16
17 // worker
18 type Worker struct {
19     requests chan Request // buffered work channel
20     pending int // count of tasks
21     index int // index in heap
22 }
23
24 func (w *Worker) work(done chan *Worker) {
25     for {
26         req := <-w.requests
27         req.c <- req.fn()
28         done <- w
29     }
30 }

```

```

31
32 // balancer
33 type Pool []*Worker
34 type Balancer struct {
35     pool Pool
36     done chan *Worker
37 }
38
39 func (b *Balancer) balance(work chan Request) {
40     for {
41         select {
42             case req := <-work: b.dispatch(req)
43             case w := <-b.done: b.completed(w)
44         }
45     }
46 }
47
48 // worker pool
49 func (p Pool) Less(i, j int) bool {
50     return p[i].pending < p[j].pending
51 }
52
53 // dispatcher
54 func (b *Balancer) dispatch(req Request) {
55     w := heap.Pop(&b.pool).(*Worker)
56     w.requests <- req
57     w.pending++
58     heap.Push(&b.pool, w)
59 }
60
61 // completed
62 func (b *Balancer) completed(w *Worker) {
63     w.pending--
64     heap.Remove(&b.pool, w.index)
65     heap.Push(&b.pool, w)
66 }

```

9 Classical Synchronisation Problems

9.1 Producer Consumer

```

1 // producer
2 Produce Item;
3 wait(notFull);
4 wait(mutex);
5 buffer[in] = item;
6 in = (in+1) % K;
7 count++;

```

```

8  signal(mutex);
9  signal(notEmpty);
10
11 // consumer
12 wait(notEmpty);
13 wait(mutex);
14 item = buffer[out];
15 out = (out+1) % K;
16 count--;
17 signal(mutex);
18 signal(notFull);
19 Consume Item;

```

9.2 Readers Writers

```

1 // lightswitch
2 counter = 0
3 mutex = Semaphore(1)
4 roomEmpty = Semaphore(1)
5 lock(semaphore):
6     mutex.wait()
7     counter += 1
8     if counter == 1:
9         semaphore.wait()
10    mutex.signal()
11 unlock(semaphore):
12    mutex.wait()
13    counter -= 1
14    if counter == 0:
15        semaphore.signal()
16    mutex.signal()
17
18 // writer
19 turnstile.wait()
20 roomEmpty.wait()
21 // CS
22 turnstile.signal()
23 roomEmpty.signal()
24
25 // reader
26 turnstile.wait()
27 turnstile.signal()
28 readLightswitch.lock(roomEmpty)
29 // CS
30 readLightswitch.unlock(roomEmpty)

```

9.3 std::barrier

```

1 std::barrier sync_point(std::ssize(x), complete);
2 auto work = [&](std::string y) {
3     // work
4     sync_point.arrive_and_wait();
5     // work
6     sync_point.arrive_and_wait();
7 }
8 std::vector<std::thread> threads;
9 for (auto const& y : x) threads.emplace_back(work, y);
10 for (auto& thread : threads) thread.join();

```

9.4 Barrier Implementation

9.5 C++

```

1 struct Barrier {
2     std::ptrdiff_t expected;
3     std::ptrdiff_t count;
4     std::mutex mut;
5     std::counting_semaphore<> turnstile;
6     std::counting_semaphore<> turnstile2;
7     Barrier(std::ptrdiff_t expected)
8         : expected{expected}, count{0}, mut{}, turnstile{0},
9           turnstile2{0} {}
10    void arrive_and_wait() {
11        {
12            std::scoped_lock lock{mut};
13            count++;
14            if (count == expected) {
15                turnstile2.acquire(); // close waiter turnstile
16                turnstile.release(expected); // open turnstile into
17                                         CS
18            }
19        }
20        turnstile.acquire();
21        {
22            std::scoped_lock lock{mut};
23            count--;
24            if (count == 0) {
25                turnstile.acquire(); // close turnstile to reset
26                                   barrier
27                turnstile2.release(expected); // open waiter
28                                   turnstile
29            }
30        }
31        turnstile2.acquire();
32    }
33 };

```

9.6 Golang

```
1 type Barrier struct {
2     wg sync.WaitGroup
3     wg2 sync.WaitGroup
4 }
5
6 func (b *Barrier) Init(expected int) {
7     b.wg.Add(expected)
8     b.wg2.Add(expected)
9 }
10
11 func (b *Barrier) Wait() {
12     b.wg.Done()
13     b.wg.Wait()
14     // all threads have called wait
15     // reset barrier
16     b.wg.Add(1)
17     b.wg2.Done()
18     // wait for barrier to fully reset
19     b.wg2.Wait()
20     // barrier reset
21     b.wg2.Add(1)
22 }
```

9.7 Dining Philosophers

9.7.1 C++

```
1 // solution 1: basic solution
2 template<size_t NumP>
3 struct DiningTable {
4     using ChpStick = std::mutex;
5     ChpStick chpSticks[NumP];
6     ChpStick& get_left_chpStick(size_t pid) { return chpSticks[pid];
7     }
8     ChpStick& get_right_chpStick(size_t pid) { return chpSticks[(pid
9     +1) % NumP]; }
10     void eat(size_t pid, void (*eat_callback)(size_t pid)) {
11         std::scoped_lock lock{get_left_chpStick(pid),
12         get_right_chpStick(pid)};
13         eat_callback(pid);
14     }
15 };
16
17 // solution 2: limited eaters
18 template<size_t NumP>
19 struct DiningTable {
20     using ChpStick = std::mutex;
```

```

18 ChpStick chpSticks[NumP];
19 std::counting_semaphore<> footman_sem{NumP-1};
20 ChpStick& get_left_chpStick(size_t pid) { return chpSticks[pid];
    }
21 ChpStick& get_right_chpStick(size_t pid) { return chpSticks[(pid
    +1) % NumP]; }
22 void eat(size_t pid, void (*eat_callback)(size_t pid)) {
23     footman_sem.acquire();
24     std::scoped_lock left_lock{get_left_chpStick(pid)};
25     std::scoped_lock right_lock{get_right_chpStick(pid)};
26     eat_callback(pid);
27     footman_sem.release();
28 }
29 };
30
31 // solution 3: fair semaphore
32 template<size_t NumP>
33 struct DiningTable {
34     using ChpStick = FairMutex;
35     ChpStick chpSticks[NumP];
36     FairSemaphore footman_sem{NumP-1};
37     ChpStick& get_left_chpStick(size_t pid) { return chpSticks[pid];
    }
38     ChpStick& get_right_chpStick(size_t pid) { return chpSticks[(pid
    +1) % NumP]; }
39     void eat(size_t pid, void (*eat_callback)(size_t pid)) {
40         footman_sem.acquire();
41         std::scoped_lock left_lock{get_left_chpStick(pid)};
42         std::scoped_lock right_lock{get_right_chpStick(pid)};
43         eat_callback(pid);
44         footman_sem.release();
45     }
46 };

```

9.7.2 Golang

```

1 type ChpStick struct {}
2 type DiningTable struct {
3     numPhilosophers int
4     chpStickChs []chan ChpStick
5 }
6
7 func (t *DiningTable) Init(numPhilosophers int) {
8     t.numPhilosophers = numPhilosophers
9     t.chpStickChs = make([]chan ChpStick, 0, numPhilosophers)
10    for i := 0; i < numPhilosophers; i++ {
11        chpStick := make(chan ChpStick, 1)
12        chpStick <- ChpStick{}

```

```

13         t.chpStickChs = append(t.chpStickChs, chpStick)
14     }
15 }
16
17 // solution 1: odd-even ring communication
18 func (t *DiningTable) Eat(pid int, eat_callback func(pid int)) {
19     evenChpStickCh := t.evenChpStickCh(pid)
20     oddChpStickCh := t.oddChpStickCh(pid)
21     <-evenChpStickCh
22     <-oddChpStickCh
23     eat_callback(pid)
24     evenChpStickCh <- ChpStick{}
25     oddChpStickCh <- ChpStick{}
26 }
27
28 func (t *DiningTable) leftChpStickCh(pid int) chan ChpStick {
29     return t.chpStickChs[pid]
30 }
31
32 func (t *DiningTable) rightChpStickCh(pid int) chan ChpStick {
33     return t.chpStickChs[(pid+1) % t.numPhilosophers]
34 }
35
36 func (t *DiningTable) evenChpStick(pid int) chan ChpStick {
37     if pid % 2 == 0 {
38         return t.leftChpStickCh(pid)
39     } else {
40         return t.rightChpStickCh(pid)
41     }
42 }
43
44 func (t *DiningTable) oddChpStick(pid int) chan ChpStick {
45     if pid % 2 == 1 {
46         return t.leftChpStickCh(pid)
47     } else {
48         return t.rightChpStickCh(pid)
49     }
50 }
51
52 // solution 2: swap order of chopsticks to avoid deadlock
53 func (t * DiningTable) Eat(pid int, eat_callback func(pid int)) {
54     evenChpStickCh := t.evenChpStickCh(pid)
55     oddChpStickCh := t.oddChpStickCh(pid)
56
57 breakLock:
58     for {
59         <-evenChpStickCh
60         select {
61             case <-oddChpStickCh:

```

```

62         break breakLock
63     default:
64         // swap
65     }
66     evenChpStickCh <- ChpStick{}
67     <-oddChpStickCh
68     select {
69         case <-evenChpStickCh:
70             break breakLock
71         default:
72             /// swap
73     }
74 }
75 eat_callback(pid)
76 evenChpStickCh <- ChpStick{}
77 oddChpStickCh <- ChpStick{}
78 }
79
80 // solution 3: limited eater using footman
81 type DiningTable struct {
82     numPhilosophers int
83     chpStickChs []chan ChpStick
84     footman chan struct{}
85 }
86
87 func (t *DiningTable) Init(numPhilosophers int) {
88     t.numPhilosophers = numPhilosophers
89     t.chpStickChs = make([]chan ChpStick, 0, numPhilosophers)
90     for i := 0; i < numPhilosophers; i++ {
91         chpStick := make(chan ChpStick, 1)
92         chpStick <- ChpStick{}
93         t.chpStickChs = append(t.chpStickChs, chpStick)
94     }
95     t.footman = make(chan struct{}, numPhilosophers - 1)
96     for i := 0; i < numPhilosophers - 1; i++ {
97         t.footman <- struct{}{}
98     }
99 }
100
101 func (t *DiningTable) Eat(pid int, eat_callback(func(pid int))) {
102     leftChpStickCh := t.leftChpStickCh(pid)
103     rightChpStickCh := t.rightChpStickCh(pid)
104     <-t.footman
105     select {
106         case <-leftChpStickCh: <-rightChpStickCh
107         case <-rightChpStickCh: <-leftChpStickCh
108     }
109     eat_callback(pid)
110     leftChpStickCh <- ChpStick{}

```

```

111     rightChpStickCh <- ChpStick{}
112     t.footman <- struct{}{}
113 }

```

9.7.3 Pseudocode

```

1  // Tanenbaum solution
2  int state[N];
3  Semaphore mutex = 1;
4  Semaphore s[N];
5
6  void philosopher(int i) {
7      while (TRUE) {
8          Think();
9          takeChpSticks(i);
10         Eat();
11         putChpSticks(i);
12     }
13 }
14
15 void takeChpSticks(i) {
16     wait(mutex);
17     state[i] = HUNGRY;
18     safeToEat(i);
19     signal(mutex);
20     wait(s[i]);
21 }
22
23 void safeToEat(i) {
24     if ((state[i] == HUNGRY) &&
25         (state[LEFT] != EATING) &&
26         (state[RIGHT] != EATING)) {
27         state[i] = EATING;
28         signal(s[i]);
29     }
30 }
31
32 void putChpSticks(i) {
33     wait(mutex);
34     state[i] = THINKING;
35     safeToEat(LEFT);
36     safeToEat(RIGHT);
37     signal(mutex);
38 }
39
40 // limited eater
41 // with N-1, guaranteed to have at least one success
42 void philosopher(int i) {

```

```

43     while (TRUE) {
44         Think();
45         wait(seats);
46         wait(chpStick[LEFT]);
47         wait(chpStick[RIGHT]);
48         Eat();
49         signal(chpStick[LEFT]);
50         signal(chpStick[RIGHT]);
51         signal(seats);
52     }
53 }

```

9.8 Barbershop

9.9 Pseudocode

```

1  // pseudocode
2  customers = 0
3  mutex = Semaphore(1)
4  customer = Semaphore(0)
5  barber = Semaphore(0)
6  customerDone = Semaphore(0)
7  barberDone = Semaphore(0)
8  // customer
9  wait(mutex);
10 if (customers == n) {
11     signal(mutex);
12     exit();
13 }
14 customers += 1;
15 signal(mutex);
16 signal(customer);
17 wait(barber);
18 getHairCut();
19 signal(customerDone);
20 wait(barberDone);
21 wait(mutex);
22 customers -= 1;
23 signal(mutex);
24 // barber
25 while (TRUE) {
26     wait(customer);
27     signal(barber);
28     cutHair();
29     wait(customerDone);
30     signal(barberDone);
31 }

```

9.10 C++

```
1  template <size_t MaxCustomers>
2  struct Barbershop {
3      size_t customers;
4      std::mutex mut;
5      std::counting_semaphore<> customer_sem;
6      std::counting_semaphore<> barber_sem;
7      std::counting_semaphore<> done_customer_sem;
8      std::counting_semaphore<> done_barber_sem;
9      void barber(void (*cutHair)()) {
10         while (true) {
11             customer_sem.acquire();
12             barber_sem.release();
13             cutHair();
14             done_customer_sem.acquire();
15             done_barber_sem.release();
16         }
17     }
18     void customer(void (*balk)(), void (*getHairCut)()) {
19         {
20             std::scoped_lock lock{mut};
21             if (customers == MaxCustomers + 1) {
22                 balk();
23                 return;
24             }
25             customers++;
26         }
27         customer_sem.release();
28         barber_sem.acquire();
29         getHairCut();
30         done_customer_sem.release();
31         done_barber_sem.acquire();
32         {
33             std::scoped_lock lock{mut};
34             customers--;
35         }
36     }
37 };
```

9.11 Golang

```
1  // might not be FIFO
2  type Barbershop struct {
3      chairs chan int
4      barberAvailable chan struct{}
5  }
6
```

```

7 func (bs *Barbershop) Init(numChairs int) {
8     bs.chairs = make(chan int, numChairs)
9     bs.barberAvailable = make(chan struct{})
10 }
11
12 func (bs *Barbershop) Barber(cutHair func()) {
13     for {
14         bs.barberAvailable <- struct{}{}
15         <-bs.chairs
16         cutHair()
17     }
18 }
19
20 func (bs *Barbershop) Customer(balk func(), getHairCut func()) {
21     select {
22     case bs.chairs <- 1:
23         <-bs.barberAvailable
24         getHairCut()
25     default:
26         balk()
27     }
28 }

```

10 Rust

10.1 Rust Basics

```

1 fn add_one(i: i32) -> i32 { i+1 }
2
3 let x = 1; // immutable
4 let mut y = 1; // mutable
5 let z: i32 = 1;
6
7 // closures
8 |i| i + 2
9 |i: i32| -> i32 { i + 2 }
10
11 &str // char array like
12 String // string type buffer
13
14 // struct and enum literals
15 OrderDetails {
16     price: 1,
17     count: 1,
18     id: 1
19 }
20
21 match order {

```

```

22     Order::Buy(details) => smth(),
23     Order::Sell(details) => smth(),
24     Order::Cancel { id } => smth()
25 }
26
27 let v = 1;
28 match v {
29     1 => ...,
30     _ => ...
31 }
32
33 pub enum Result<T, E> {
34     Ok(T),
35     Err(E),
36 }
37
38 // optionals
39 pub enum Option<T> {
40     None,
41     Some(T),
42 }
43
44 let order_id = Some(3);
45 let Some(value) = order_id else {
46     // panic
47 };
48
49 order_id.unwrap();
50
51 // ? operator unwraps 'Ok' or returns 'Err'
52 let content = std::fs::read_to_string("config.txt"?);
53 let content = match std::fs::read_to_string("config.txt") { Ok(v) =>
    v, Err(e) => return Err(e.into()) };
54
55 // vectors
56 let mut odd = Vec::new();
57 odd.push(1);
58 let numbers = (1..=100).collect::<Vec<u32>>();
59 let even: Vec<u32> = numbers.iter().filter(|&i| i % 2 == 0).collect
    ();
60 for n in &odd { } // borrow in loop

```

10.2 Ownership and Borrowing

```

1 // transferring ownership
2 fn main() {
3     let mut book = Vec::new();
4     book.push(...);

```

```

5     book.push(...);
6     publish(book);
7     // ownership transferred, shallow copy of fields to other stack
8     // original book cannot be used
9     // publish(book); // compilation error, used of moved value
10 }
11 fn publish(book: Vec<String>) {
12     // ...
13 } // book gets dropped here
14
15 // immutable borrow, shared reference
16 fn main() {
17     let mut book = Vec::new();
18     book.push(...);
19     book.push(...);
20     publish(&book);
21     // reference borrowed
22     publish(&book); // ok now
23     // book is mutable here
24 }
25 fn publish(book: &Vec<String>) {
26     // book is immutable here and in main
27 }
28
29 // mutable borrow
30 fn main() {
31     let mut book = Vec::new();
32     book.push(...);
33     book.push(...);
34     publish(&mut book);
35     publish(&mut book);
36 }
37 // book doesn't need mut since itself doesn't need to be mutable
38 // the underlying reference is already mutable
39 fn publish(book: &mut Vec<String>) {
40     // book is not usable in main at all
41     book.push(...)
42 }
43
44 // return ownership
45 fn publish(book: Vec<String>) {
46     book
47 }

```

10.3 Threads

```

1 let loc = thread::spawn(|| { // anonymous function syntax
2     "world"

```

```

3 });
4 println!("Hello, {}!", loc.join().unwrap());
5
6 // move transfers ownership
7 // without move, rust will try to infer (not recommended)
8 let mut dst = Vec::new();
9 thread::spawn(move || {
10     dst.push(3);
11 });
12 dst.push(3); // error, ownership was transferred

```

10.4 Lifetimes

```

1 // static lifetime, data does not borrow anything that could go out
  // of scope
2 let s: &'static str = "hello";
3
4 // annotations
5 // x, y are valid for the lifetime of scope a
6 // returned reference will live at least as long as the shortest
  // lifetime of x, y
7 fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
8     if x.len() > y.len() {
9         x
10    } else {
11        y
12    }
13 }
14
15 // 2 different scopes
16 // z has different lifetime compared to x, y
17 fn longest_print_z<'a, 'b>(x: &'a str, y: &'a str, z: 'b str) -> &'a
  str {
18     println!("{}", z);
19     if x.len() > y.len() {
20         x
21    } else {
22        y
23    }
24 }
25
26 // reference counted type
27 // error, cannot be sent across threads
28 let v = Rc::new(vec![1, 2, 3]);
29 let v2 = v.clone();
30 thread::spawn(move || {
31     println!("{}", v.len());
32 });

```

```

33 another_fn(v2.clone());
34
35 // traits (interfaces)
36 trait Clone {
37     fn clone(&self) -> Self;
38 }
39 impl<T: Clone> Clone for Vec<T> {
40     fn clone(&self) -> Vec<T> {
41         let mut v = Vec::new();
42         for elem in self {
43             v.push(elem.clone());
44         }
45         return v;
46     }
47 }
48
49 // atomic reference counted
50 // only allows shared references
51 let v = Arc::new(vec![1, 2, 3]);
52 let v2 = v.clone();
53 // move shared reference into thread
54 thread::spawn(move || {
55     // v cannot be modified
56     println!("{}", v.len());
57 });
58 another_fn(&v2);

```

10.5 Rust Synchronisation

```

1 // mutex
2 fn sync_inc(counter: &Mutex<i32>) {
3     let mut data MutexGuard<i32> = counter.lock();
4     *data += 1;
5 }
6
7 // atomics
8 let number = AtomicUsize::new(10);
9 let prev = number.fetch_add(1, SeqCst);
10 assert_eq!(prev, 10);
11 let prev = number.swap(2, SeqCst);
12 assert_eq!(prev, 11);
13 assert_eq!(number.load(SeqCst), 2);
14
15 // MPSC FIFO queue
16 let (tx, rx) = mpsc::channel();
17 let tx2 = tx.clone();
18 thread::spawn(move || tx.send(5));
19 thread::spawn(move || tx2.send(4));

```

```

20 // order of printing is not specified
21 println!("{:?}", rx.recv());
22 println!("{:?}", rx.recv());

```

10.6 Crossbeam

```

1 // scoped threads
2 fn main() {
3     let v = vec![1, 2, 3];
4     std::thread::scope(|scope| {
5         scope.spawn(|inner_scope| {
6             // do stuff with v
7         });
8     }).unwrap();
9     // v is back here
10 }
11
12 // producer consumer
13 fn main() {
14     let (send_end, receive_end) = bounded(CHANNEL_CAPACITY);
15     let mut threads = vec![];
16     for _i in 0 .. NUM_THREADS {
17         let send_end = send_end.clone();
18         threads.push(
19             thread::spawn(move || {
20                 for _k in 0 .. ITEMS_PER_THREAD {
21                     let produced_value = produce_item();
22                     send_end.send(produced_value).unwrap();
23                 }
24             })
25         )
26     }
27     for _j in 0 .. NUM_THREADS {
28         let receive_end = receive_end.clone();
29         threads.push(
30             thread::spawn(move || {
31                 for _k in 0 .. ITEMS_PER_THREAD {
32                     let to_consume = receive_end.recv().unwrap();
33                     consume_item(to_consume);
34                 }
35             })
36         )
37     }
38     for t in threads {
39         let _ = t.join();
40     }
41 }
42

```

```

43 // exponential backoff
44 use crossbeam_utils::Backoff;
45 use std::sync_atomic::AtomicUsize;
46 use std::sync::atomic::Ordering::SeqCst;
47 fn fetch_mul(a: &AtomicUsize, b: usize) -> usize {
48     let backoff = Backoff::new();
49     loop {
50         let val = a.load(SeqCst);
51         if a.compare_and_swap(val, val.wrapping_mul(b), SeqCst) ==
            val {
52             return val;
53         }
54         backoff.spin();
55     }
56 }
57
58 // rough implementation of spin wait
59 fn spin_wait(ready: &AtomicBool) {
60     let backoff = Backoff::new();
61     while !ready.load(SeqCst) {
62         backoff.snooze();
63     }
64 }

```

10.7 Rayon

```

1 // sequential maximum
2 fn main() {
3     let vec = init_vector();
4     let max = MIN;
5     vec.iter().for_each(|n| {
6         if *n > max {
7             max = *n;
8         }
9     });
10 }
11 // with Rayon
12 fn main() {
13     let vec = init_vector()
14     let max = AtomicI64::new(MIN);
15     vec.par_iter().for_each(|n| {
16         loop {
17             let old = max.load(Ordering::SeqCst);
18             if *n <= old {
19                 break;
20             }
21             let returned = max.compare_and_swap(old, *n, Ordering::
                SeqCst);

```

```

22         if returned == old {
23             break;
24         }
25     }
26 });
27 }

```

10.8 Futures

```

1  // simplified trait
2  trait Future {
3      type Output;
4      fn poll(&mut self, cx: &mut Context) -> Poll<Self::Output>;
5  }
6  enum Poll<T> {
7      Ready(T),
8      Pending,
9  }
10
11 // composition
12 let future = placeOnStove(meat)
13     .then(|meat| cookOneSide(meat))
14     .then(|meat| flip(meat))
15     .then(|meat| cookOneSide(meat));
16
17 // clunky composition
18 fn addToInbox(email_id: u64, recipient_id: u64) -> impl Future<
19     Output=Result<(), Error>> {
20     loadMessage(email_id)
21         .and_then(|message| get_recipient(message, recipient_id))
22         .map(|(message, recipient)| recipient.verifyHasSpace(&
23             message))
24         .and_then(|(message, recipient)| recipient.addToInbox(
25             message))
26 }
27
28 // async, returns a future
29 // await waits for a future and retrieves a value
30 async fn addToInbox(email_id: u64, recipient_id: u64) -> Result<(),
31     Error> {
32     let message = loadMessage(email_id).await?;
33     let recipient = get_recipient(recipient_id).await?;
34     recipient.verifyHasSpace(&message)?;
35     recipient.addToInbox(message).await
36 }
37
38 // state management
39 enum AddToInboxState {

```

```

36     NotYetStarted { email_id: u64, recipient_id: u64},
37     WaitingLoadMessage { recipient_id: u64, state: LoadMessageFuture
38         },
39     WaitingGetRecipient { message: Message, state:
40         GetRecipientFuture },
41     WaitingAddToInbox { state: AddToInboxFuture },
42     Completed { result: Result<(), Error> },
43 }
44
45 fn poll(*) {
46     match self.state {
47         NotYetStarted(email_id, recipient_id) => {
48             let next_future = load_message(email_id);
49             // switch to WaitingLoadMessage state
50         },
51         WaitingLoadMessage(email_id, recipient_id, state) => {
52             match state.poll() {
53                 Ready(message) => {
54                     let next_future = get_recipient(recipient_id);
55                     // switch to WaitingGetRecipient state
56                 },
57                 Pending => return Pending,
58             }
59         },
60         WaitingGetRecipient(message, recipient_id, state) => {
61             match state.poll() {
62                 Ready(recipient) => {
63                     recipient.verifyHasSpace(&message)?;
64                     let next_future = recipient.addToInbox(message);
65                     // switch to WaitingAddToInbox state
66                 },
67                 Pending => return Pending,
68             }
69         },
70         // ...
71     }
72 }

```

11 TLA+

11.1 Basics

```

1 // AND: /\
2 // OR: \/
3 // NOT: ~a
4
5 // predicate: expression returning boolean
6 operator(a, b, c) == (a /\ b) \/ (a /\ ~c)

```

```

7
8 // actions: tests
9 // tick means after
10 Next ==
11     state = "hello"
12     /\ state' = "goodbye"
13
14 // counting
15 Init ==
16     x = 1
17 Next ==
18     (x = 1 /\ x' = 2)
19     \/ (x = 2 /\ x' = 3)
20 // alternatively, we can explicitly name the steps known as
    operators
21 // deadlocks at state 3
22 // staying at a state is not valid
23 Init == x = 1
24 Step1 == x = 1 /\ x' = 2
25 Step2 == x = 2 /\ x' = 3
26 Done == x = 3 /\ UNCHANGED x
27 // add new valid transitions
28 Next == Step1 \/ Step2 \/ Done \/ UNCHANGED x
29
30 // properties
31 [] (x > 0) // x always > 0
32 <> (x = 2) // eventually x = 2
33 <>[] (x = 3) // eventually x always = 3
34 (x = 2) ~> (x = 3) // x = 2 leads to x = 3
35 // stuttering causes these to fail
36
37 Spec = Init /\ [] (Next \/ UNCHANGED x) // refactor specification
38 Spec = Init /\ [] [Next]_x // syntax for next or stutter
39 Spec = Init /\ [] [Next]_x /\ WF_x(Next) // fairness, if next step is
    available, eventually try it, ignoring stutter
40
41 // set theory
42 e \in S // inclusion
43 {1, 2, 3} // define set
44 {e \in S : p} // define set using predicate
45 \A e \in S : p // for all elements check predicate
46 \E x \in S : p // there exists element satisfying predicate

```

11.2 Producer Consumer

```

1 // producer states and actions
2 CheckWritable ==
3     producerState = "ready"

```

```

4      /\ queueSize < MaxQueueSize
5      /\ producerState' = "canWrite"
6      /\ UNCHANGED queueSize
7
8  Write ==
9      producerState = "canWrite"
10     /\ producerState' = "ready"
11     /\ queueSize' = queueSize + 1
12
13  ProducerAction == CheckWritable \/ Write
14
15  // consumer states and actions
16  CheckReadable ==
17      consumerState = "ready"
18      /\ queueSize > 0
19      /\ consumerState' = "canRead"
20      /\ UNCHANGED queueSize
21
22  Read ==
23      consumerState = "canRead"
24      /\ consumerState' = "ready"
25      /\ queueSize' = queueSize - 1
26
27  ConsumerAction == CheckReadable \/ Read
28
29  // complete script
30  VARIABLES
31      queueSize,
32      producerState,
33      consumerState
34
35  MaxQueueSize == 2
36
37  Init ==
38      queueSize = 0
39      /\ producerState = "ready"
40      /\ consumerState = "ready"
41
42  Next ==
43      ProducerAction
44      \/ ConsumerAction
45      \/ (UNCHANGED producerState
46          /\ UNCHANGED consumerState
47          /\ UNCHANGED queueSize)
48
49  // properties
50  AlwaysWithinBounds ==
51      [] (queueSize >= 0
52          /\ queueSize <= MaxQueueSize)

```

```

53
54 // multiple producers and consumers
55 // actually fails checks, since check and write a separate actions
56 CONSTANT producers, consumers
57 producers = {"p1", "p2"}
58 consumers = {"c1", "c2"}
59 VARIABLES queueSize, producerState, consumerState // maps /
    dictionaries
60 MaxQueueSize == 2
61 Init ==
62     queueSize = 0
63     /\ producerState = [p \in producers |-> "ready"]
64     /\ consumerState = [c \in consumers |-> "ready"]
65
66 CheckWritable(p) ==
67     producerState[p] = "ready"
68     /\ queueSize < MaxQueueSize
69     /\ producerState' =
70         [producerState EXCEPT ![p] = "canWrite"]
71     /\ UNCHANGED queueSize
72     /\ UNCHANGED consumerState
73
74 Write(p) ==
75     producerState[p] = "canWrite"
76     /\ queueSize' = queueSize + 1
77     /\ producerState' =
78         [producerState EXCEPT ![p] = "ready"]
79     /\ UNCHANGED consumerState
80
81 ProducerAction ==
82     \E p \in producers : CheckWritable(p) /\ Write(p)
83
84 CheckReadable(c) ==
85     consumerState[c] = "ready"
86     /\ queueSize > 0
87     /\ consumerState' =
88         [consumerState EXCEPT ![c] = "canRead"]
89     /\ UNCHANGED queueSize
90     /\ UNCHANGED producerState
91
92 Read(c) ==
93     consumerState[c] = "canRead"
94     /\ queueSize' = queueSize - 1
95     /\ consumerState' =
96         [consumerState EXCEPT ![c] = "ready"]
97     /\ UNCHANGED producerState
98
99 ConsumerAction ==
100     \E c \in consumers : CheckReadable(c) /\ Read(c)

```

12 Finals Tutorial Stuff

12.1 Go Concurrent Counter

```
1 // worker
2 count := <-ch
3 count++
4 wg.Done()
5 ch <- count
6
7 // master
8 wg.Wait()
9 fmt.Println(<-ch)
```

12.2 Go Multi Producer Multi Consumer Queue

```
1 func producer(done chan struct{}, q chan int) {
2     for {
3         select {
4             case q <- 1:
5             case <-done:
6                 return
7         }
8     }
9 }
10
11 func consumer(done chan struct{}, q chan int, sumCh chan<- int) {
12     sum := 0
13     for {
14         select {
15             case <-done:
16                 sumCh <- sum
17                 close(sumCh)
18                 return
19             case num := <-q:
20                 sum += num
21         }
22     }
23 }
24
25
26 var (
27     NumProducer = 5
28     NumConsumer = 5
29 )
30
31 func main() {
32     start, done := make(chan struct{}), make(chan struct{})
```

```

33     q := make(chan int)
34
35     // 1 channer per consumer
36     sumChs := make([]chan int, 0, NumConsumer)
37     for i := 0; i < NumConsumer; i++ {
38         sumCh := make(chan int, 1)
39         sumChs = append(sumChs, sumCh)
40     }
41
42     for i := 0; i < NumProducer; i++ {
43         go func() {
44             <-start
45             producer(done, q)
46         }()
47     }
48     for j := 0; j < NumConsumer; j++ {
49         j := j // capture j in the scope, for older Go versions
50         go func() {
51             <-start
52             consumer(done, q, sumChs[j])
53         }()
54     }
55
56     close(start) // start goroutines
57     time.Sleep(time.Second) // run for 1 second
58     close(done) // stop goroutines
59
60     // collect all sums
61     sum := 0
62     for _, ch := range sumChs { // NOTE: range over slice, not
63         combined channel
64         sum += <-ch
65     }
66     fmt.Println("Sum: ", sum)
67 }

```

12.3 Go Contexts

```

1 // using a context
2 func producer(ctx context.Context, q queue) {
3     for {
4         select {
5             case q<- 1:
6             case <-ctx.Done(): return
7         }
8     }
9 }
10 // creating contexts

```

```

11 ctx, _ := context.WithTimeout(context.Background(), 4 * time.Second)
12 // context tree, if parent is cancelled, child is also cancelled
13 ctx2, _ := context.withTimeout(ctx, 2 * time.Second)
14 // manually cancel a context
15 ctx3, cancel := context.WithCancel(ctx)
16 go func() { cancel() }()

```

12.4 Go Fan-In Fan-Out

```

1 type worker struct {
2     inputCh  chan Event
3     outputCh chan Event
4 }
5
6 func newWorker() *worker {
7     return &worker{
8         inputCh:  make(chan Event),
9         outputCh: make(chan Event),
10    }
11 }
12
13 // workers with promises
14 func (w *worker) start(
15     ctx context.Context,
16     fn EventFunc, workerQueue chan *worker, wg *sync.WaitGroup,
17 ) {
18     wg.Add(1)
19     go func() {
20         defer wg.Done()
21         for {
22             select {
23             case <-ctx.Done():
24                 return
25             case workerQueue <- w:
26                 e := <-w.inputCh
27                 w.outputCh <- fn(e)
28             }
29         }
30     }()
31 }
32
33 // ordered multiplexer
34 func orderedMux(
35     ctx context.Context, cancel context.CancelFunc,
36     inputCh chan Event, workerQueue chan *worker, workerOutputCh
37     chan chan Event,
38 ) {
39     go func() {

```

```

39         for {
40             select {
41                 case <-ctx.Done():
42                     return
43                 case e := <-inputCh:
44                     w := <-workerQueue
45                     w.inputCh <- e
46                     workerOutputCh <- w.outputCh
47             }
48         }
49     }()
50 }
51
52 // in main
53 readerDone := make(chan struct{})
54 go func() {
55     for outputCh := range workerOutputCh { // <-- reading a stream
56         of promises
57         output, more := <-outputCh // <-- read from the promise once
58         only
59         if !more {
60             break // <-- a worker breaks its promise; workers
61             exiting
62         }
63         fmt.Printf("Event id: %d\n", output.id)
64     }
65     close(readerDone)
66 }()
67
68 // serialiser
69 finalOutputCh := make(chan Event, 1)
70 go func() {
71     buffer := make(map[int64]Event)
72     var nextEvent int64 = 1
73     for output := range processedCh {
74         buffer[output.id] = output
75         for {
76             if event, exists := buffer[nextEvent]; exists {
77                 finalOutputCh <- event
78                 nextEvent += 1
79             } else {
80                 break
81             }
82         }
83     }
84     close(finalOutputCh)
85 }()

```

12.5 Go Pipelining

```
1 func toPipelineStage(  
2     ctx context.Context,  
3     reqCh <-chan Request, instances int,  
4     fn func(*Request) *Request,  
5 ) chan Request {  
6     outCh := make(chan Request)  
7     for i := 0; i < instances; i++ {  
8         go func() {  
9             for req := range reqCh {  
10                 select {  
11                     case outCh <- *fn(&req):  
12                     case <-ctx.Done():  
13                         return  
14                 }  
15             }  
16         }()  
17     }  
18     return outCh  
19 }  
20  
21 // in main  
22 c := make(chan Request, 100)  
23 c = toPipelineStage(ctx, c, 2, step1)  
24 c = toPipelineStage(ctx, c, 5, step2)  
25 c = toPipelineStage(ctx, c, 10, step3)  
26 c = toPipelineStage(ctx, c, 5, step4)
```

12.6 C++ H2O using Barrier

```
1 std::counting_semaphore<> oxygenSem{1};  
2 std::counting_semaphore<> hydrogenSem{2};  
3 std::barrier<> barrier{3};  
4  
5 void oxygen(void (*bond)()) {  
6     oxygenSem.acquire();  
7     barrier.arrive_and_wait();  
8     bond();  
9     oxygenSem.release();  
10 }  
11  
12 void hydrogen(void (*bond)()) {  
13     hydrogenSem.acquire();  
14     barrier.arrive_and_wait();  
15     bond();  
16     hydrogenSem.release();  
17 }
```

12.7 Go H2O using Daemon

```
1 type WaterFactoryWithDaemon struct {
2     // Channels for atoms to send their arrival requests
3     precomH chan chan struct{}
4     precomO chan chan struct{}
5 }
6
7 func NewFactoryWithDaemon() WaterFactoryWithDaemon {
8     wfd := WaterFactoryWithDaemon{
9         precomH: make(chan chan struct{}),
10        precomO: make(chan chan struct{}),
11    }
12
13    // Daemon goroutine
14    go func() {
15        for {
16            // Step 1: (Precommit)
17            // Receive arrival requests from 2 hydrogen and 1 oxygen
18            // atoms
19            h1 := <-wfd.precomH
20            h2 := <-wfd.precomH
21            o := <-wfd.precomO
22            // Step 2: (Commit)
23            // Tell the 3 atoms to start bonding
24            h1 <- struct{}{}
25            h2 <- struct{}{}
26            o <- struct{}{}
27            // Step 3: (Postcommit)
28            // Wait until the 3 atoms have finished before looping
29            // We re-use the same communication channel as (Commit)
30            <-h1
31            <-h2
32            <-o
33        }
34    }()
35
36    return wfd
37 }
38
39 func (wfd *WaterFactoryWithDaemon) hydrogen(bond func()) {
40     commit := make(chan struct{}) // Step 1: Create private
41     // communication channel
42     wfd.precomH <- commit          // Step 2: (Precommit)
43     <-commit                        // Step 3: (Commit)
44     bond()                         // Step 4: Bond
45     commit <- struct{}{}          // Step 5: (Postcommit)
46 }
```

```

46 func (wfd *WaterFactoryWithDaemon) oxygen(bond func()) {
47     commit := make(chan struct{}) // Step 1: Create private
        communication channel
48     wfd.precom0 <- commit          // Step 2: (Precommit)
49     <-commit                        // Step 3: (Commit)
50     bond()                         // Step 4: Bond
51     commit <- struct{}{}          // Step 5: (Postcommit)
52 }

```

12.8 Go H2O using Oxygen Leaders

```

1 type WaterFactoryWithLeader struct {
2     oxygenMutex chan struct{}
3     precomH      chan chan struct{}
4 }
5
6 func NewFactoryWithLeader() WaterFactoryWithLeader {
7     wf := WaterFactoryWithLeader{
8         oxygenMutex: make(chan struct{}, 1),
9         precomH:      make(chan chan struct{}),
10    }
11    wf.oxygenMutex <- struct{}{}
12    return wf
13 }
14
15 func (wf *WaterFactoryWithLeader) hydrogen(bond func()) {
16     commit := make(chan struct{}) // Step 1: Create private
        communication channel
17     wf.precomH <- commit          // Step 2: (Precommit)
18     <-commit                        // Step 3: (Commit)
19     bond()                         // Step 4: Bond
20     commit <- struct{}{}          // Step 5: (Postcommit)
21 }
22
23 func (wf *WaterFactoryWithLeader) oxygen(bond func()) {
24     // Step 1: Become leader
25     <-wf.oxygenMutex // For fun, we can use a channel as a mutex
26     // Step 2: (Precommit)
27     // Receive arrival requests from 2 hydrogen atoms
28     h1 := <-wf.precomH
29     h2 := <-wf.precomH
30     // Step 3: (Commit)
31     // Tell the 2 hydrogen atoms to start bonding
32     h1 <- struct{}{}
33     h2 <- struct{}{}
34     // Step 4: Bond
35     bond()
36     // Step 5: (Postcommit)

```

```

37 // Wait until the 2 hydrogen atoms to finish
38 // We re-use the same communication channel as (Commit)
39 <-h1
40 <-h2
41 // Step 6: Step down from being leader
42 wf.oxygenMutex <- struct{}{}
43 }
44
45 // H2O2 problem
46 // other oxygen is follower who does not get oxygen mutex
47 case h2o2f.precom0 <- commit: // Follower
48     <-commit // Step 3: (Commit)
49     bond() // Step 4: Bond
50     commit <- struct{}{} // Step 5: (Postcommit)

```

12.9 C++ FIFO Semaphore using Ticket Queue

```

1 struct FIFOSemaphore {
2     std::mutex mut;
3     std::condition_variable cond;
4     std::atomic<std::ptrdiff_t> next_ticket;
5     std::ptrdiff_t now_serving;
6
7     FIFOSemaphore(std::ptrdiff_t initial_count)
8         : mut{}, cond{}, next_ticket{1}, now_serving{initial_count} {}
9
10    void acquire() {
11        std::ptrdiff_t my_ticket =
12            next_ticket.fetch_add(1, std::memory_order_relaxed);
13        std::unique_lock lock{mut};
14        cond.wait(lock, [=, this]() { return now_serving >= my_ticket;
15            });
16    }
17
18    void release() {
19        {
20            std::scoped_lock lock{mut};
21            now_serving++;
22        }
23        cond.notify_all();
24    };

```

12.10 C++ FIFO Semaphore using Queue of Semaphores

```

1 struct FIFOSemaphore {
2     struct Waiter {

```

```

3     std::binary_semaphore sem{0};
4 };
5
6     std::mutex mut;
7     std::queue<std::shared_ptr<Waiter>> waiters;
8     std::ptrdiff_t count;
9
10    FIFOSemaphore(std::ptrdiff_t initial_count)
11        : mut{}, waiters{}, count{initial_count} {}
12
13    void acquire() {
14        auto waiter = std::make_shared<Waiter>();
15        {
16            std::scoped_lock lock{mut};
17            if (count > 0) {
18                count--; // Positive count,
19                return; // simply decrement without blocking
20            }
21            waiters.push(waiter); // Zero count, add to waiters
22        }
23        waiter->sem.acquire(); // and block on the semaphore
24    }
25
26    void release() {
27        std::shared_ptr<Waiter> waiter;
28        {
29            std::scoped_lock lock{mut};
30            if (waiters.empty()) {
31                count++; // No waiters, simply increment count
32                return;
33            }
34
35            waiter = waiters.front(); // Pop a waiter
36            waiters.pop();
37        }
38        waiter->sem.release(); // and signal it
39    }
40 };

```

12.11 Go FIFO Semaphore using Buffered Channel

```

1 type Semaphore struct {
2     sem chan struct{}
3 }
4
5 func NewSemaphore(capacity int, initial_count int) *Semaphore {
6     sem := Semaphore{
7         sem: make(chan struct{}, capacity),

```

```

8     }
9     for i := 0; i < initial_count; i++ {
10         sem.Release()
11     }
12     return &sem
13 }
14
15 func (s *Semaphore) Acquire() {
16     // Receive from the channel to decrement the number of items
17     // Blocks if there are no items (count is 0)
18     <-s.sem
19     // Blocked goroutines will be unblocked in FIFO order as of Go
20     // 1.24
21 }
22
23 func (s *Semaphore) Release() {
24     // Send to the channel to increment the number of items
25     s.sem <- struct{}{}
26 }

```

12.12 Go FIFO Semaphore using Daemon

```

1 type Semaphore struct {
2     acquireCh chan chan struct{}
3     releaseCh  chan struct{}
4 }
5
6 func NewSemaphore(initial_count int) *Semaphore {
7     sem := new(Semaphore)
8     sem.acquireCh = make(chan chan struct{}, 100)
9     sem.releaseCh = make(chan struct{}, 100)
10
11     go func() {
12         count := initial_count
13         // The FIFO queue that stores the channels used to unblock
14         // waiters
15         waiters := NewChanQueue()
16
17         for {
18             select {
19                 case <-sem.releaseCh: // Increment or unblock a waiter
20                     if waiters.Len() > 0 {
21                         ch := waiters.Pop()
22                         ch <- struct{}{} // Unblocks the oldest waiter
23                     } else {
24                         count++
25                     }
26                 }
27             }
28         }
29     }()
30 }

```

```

26         case ch := <-sem.acquireCh: // Decrement or add a waiter
27             if count > 0 {
28                 count--
29                 ch <- struct{}{} // Don't keep waiter blocked
30             } else {
31                 waiters.PushBack(ch) // Add waiter to back of
32                                     queue
33             }
34         }
35     }()
36
37     return sem
38 }
39
40 func (s *Semaphore) Acquire() {
41     ch := make(chan struct{})
42     // Send daemon a channel that can be used to unblock us
43     s.acquireCh <- ch
44     // Block until daemon decides to unblock us
45     <-ch
46 }
47
48 func (s *Semaphore) Release() {
49     s.releaseCh <- struct{}{}
50 }

```

12.13 Rust Concurrent Counter

```

1 // mutex solution
2 use std::thread;
3 use std::sync::Mutex;
4
5 fn main() {
6     let counter = Mutex::new(0);
7
8     thread::scope(|s| {
9         s.spawn(|| {
10             // THE FOLLOWING IS NOT IDIOMATIC RUST
11             // (what *counter.lock().unwrap() += 1 actually does)
12             use std::ops::DerefMut;
13
14             let lock_result = counter.lock();
15             let mut lock_guard = lock_result.unwrap();
16             let counter_ref: &mut i32 = lock_guard.deref_mut();
17             *counter_ref += 1;
18             // END UNIDIOMATIC RUST
19         });

```

```

20     s.spawn(|| { *counter.lock().unwrap() += 1; });
21 });
22
23 println!("{}", *counter.lock().unwrap());
24 }
25
26 // using Arc
27 use std::thread;
28 use std::sync::{Mutex, Arc};
29
30 fn main() {
31     let counter = Arc::new(Mutex::new(0));
32
33     let t0 = {
34         let counter = counter.clone();
35         thread::spawn(move || {
36             // THE FOLLOWING IS NOT IDIOMATIC RUST
37             use std::ops::{Deref, DerefMut};
38
39             let mutex: &Mutex<i32> = counter.deref();
40             let lock_result = mutex.lock();
41             let mut lock_guard = lock_result.unwrap();
42             let counter_ref: &mut i32 = lock_guard.deref_mut();
43             *counter_ref += 1;
44             // END UNIDIOMATIC RUST
45         })
46     };
47     let t1 = {
48         let counter = counter.clone();
49         thread::spawn(move || {
50             *counter.lock().unwrap() += 1;
51         })
52     };
53
54     t0.join().unwrap();
55     t1.join().unwrap();
56
57     println!("{}", *counter.lock().unwrap());
58 }
59
60 // atomics
61 use std::sync::atomic::{AtomicI32, Ordering};
62 use std::thread;
63
64 use std::sync::{Mutex, Arc};
65
66 fn main() {
67     let counter = AtomicI32::new(0);
68

```

```

69     thread::scope(|s| {
70         s.spawn(|| {
71             counter.fetch_add(1, Ordering::Relaxed);
72         });
73         s.spawn(|| {
74             counter.fetch_add(1, Ordering::Relaxed);
75         });
76     });
77
78     println!("{}", counter.load(Ordering::Relaxed));
79 }

```

12.14 Rust Line Echo Server

```

1  use std::net::SocketAddr;
2
3  use tokio::io::{AsyncBufReadExt, AsyncWriteExt, BufReader};
4  use tokio::net::{TcpListener, TcpStream};
5
6  async fn handle_client(stream: TcpStream) -> std::io::Result<()> {
7      let mut reader = BufReader::new(stream);
8      let mut buf: Vec<u8> = Vec::new();
9      loop {
10         let size = reader.read_until(b'\n', &mut buf).await?;
11         if size == 0 || buf[size - 1] != b'\n' {
12             break;
13         }
14         reader.get_mut().write_all(&buf[..size]).await?;
15         buf.clear();
16     }
17     Ok(())
18 }
19
20 #[tokio::main]
21 async fn main() -> std::io::Result<()> {
22     let port = std::env::args()
23         .nth(1)
24         .map(|s| s.parse().unwrap())
25         .unwrap_or(50000u16);
26     let listener = TcpListener::bind(SocketAddr::from(([127, 0, 0,
27         1], port))).await?;
28     loop {
29         let (socket, _) = listener.accept().await?;
30         tokio::spawn(async move {
31             eprintln!("Accepted connection");
32             std::mem::drop(handle_client(socket).await);
33             eprintln!("Connection ended");
34         });
35     }
36 }

```

```
34     }
35 }
```

12.15 Rust H2O

```
1  #![allow(dead_code)]
2
3  use futures::future::join_all;
4  use std::sync::Arc;
5  use tokio::sync::{Barrier, Semaphore};
6
7  fn bond(s: &str) {
8      println!("bond {s}!");
9  }
10
11 #[tokio::main]
12 async fn main() {
13     let n = 100;
14     let f = Arc::new(WaterFactory::new());
15
16     let hs = (0..n * 2).map(|i| {
17         let f = f.clone();
18         tokio::spawn(async move {
19             f.hydrogen(|| bond(&format!("h{i}"))).await;
20         })
21     });
22     let os = (0..n).map(|i| {
23         let f = f.clone();
24         tokio::spawn(async move {
25             f.oxygen(|| bond(&format!("o{i}"))).await;
26         })
27     });
28
29     join_all(Iterator::chain(hs, os)).await;
30 }
31
32 struct WaterFactory {
33     o_sem: Semaphore,
34     h_sem: Semaphore,
35     barrier: Barrier,
36 }
37
38 impl WaterFactory {
39     fn new() -> Self {
40         Self {
41             o_sem: Semaphore::new(1),
42             h_sem: Semaphore::new(2),
43             barrier: Barrier::new(3),

```

```

44     }
45 }
46
47 async fn oxygen(&self, bond: impl FnOnce()) {
48     let _permit = self.o_sem.acquire().await.unwrap();
49     self.barrier.wait().await;
50     bond();
51 }
52
53 async fn hydrogen(&self, bond: impl FnOnce()) {
54     let _permit = self.h_sem.acquire().await.unwrap();
55     self.barrier.wait().await;
56     bond();
57 }
58 }

```

13 PYP Questions

13.1 22/23 io_uring in C++

```

1 struct Request {
2     int client_fd;
3     data_t data;
4     res_t result;
5 };
6
7 class ConcurrentRing {
8 private:
9     LockFreeQueue<Request> queue;
10    std::counting_semaphore write{SIZE};
11    std::counting_semaphore read{0};
12 public:
13    void submit_request(Request req) {
14        write.acquire();
15        queue.push(req);
16        read.release();
17    }
18    Request retrieve_request() {
19        read.acquire();
20        std::optional<Request> req;
21        while (true) {
22            req = queue.try_pop();
23            if (req) break;
24        }
25        write.release();
26        return req.value();
27    }
28 };

```

```

29
30 int main() {
31     // Set up TCP socket
32     int server_fd = socket(AF_INET, SOCK_STREAM, 0);
33     // Bind socket to port
34     listen(server_fd, 5);
35     ConcurrentRing SQ, CQ;
36     for (int i = 0; i < W; i++) {
37         std::thread([&]() {
38             while (true) {
39                 Request req = SQ.retrieve_req();
40                 req.process();
41                 CQ.submit_request(req);
42             }
43         });
44         std::thread([&]() {
45             while (true) {
46                 Request req = CQ.retrieve_req();
47                 send(req.client_fd, req);
48             }
49         });
50     }
51     while (true) {
52         // Accept new client connection
53         int client_fd = accept(server_fd, nullptr, nullptr);
54         std::thread([&]() {
55             data_t data = read(client_fd);
56             while (data) {
57                 Request req{client_fd, data};
58                 SQ.submit_request(req);
59                 data = read(client_fd);
60             }
61         });
62     }
63 }

```

13.2 22/23 io_uring in Go

```

1 func main() {
2     SQ = make(chan Request, SIZE)
3     CQ = make(chan Request, SIZE)
4     for i := 0; i < W; i++ {
5         go func() {
6             for {
7                 select {
8                     case req := <-SQ:
9                         req.process()
10                    CQ <- req

```

```

11         default:
12     }
13 }
14 }()
15 }
16 for i := 0; i < W; i++ {
17     go func() {
18         for {
19             select {
20                 case req := <-CQ:
21                     req.queue <- req
22                 default:
23             }
24         }
25     }()
26 }
27 listener, err := net.Listen("tcp", ":8080")
28 if err != nil {
29     // handle error
30 }
31 for {
32     conn, err := listener.Accept()
33     if err != nil {
34         // handle error
35     }
36     queue := make(chan Request)
37     go func() {
38         for {
39             req := conn.read()
40             req.queue = queue
41             SQ <- req
42         }
43     }()
44     go func() {
45         for {
46             select {
47                 case req := <-queue:
48                     conn.send(req);
49                 default:
50             }
51         }
52     }()
53 }
54 }

```

13.3 22/23 io_uring in Rust

```

1 async fn main() -> Result<(), Box<dyn Error>> {

```

```

2 let addr = SocketAddr::from(([127, 0, 0, 1], 8080));
3 let listener = TcpListener::bind(&addr).await?;
4 let (sq_tx, mut sq_rx) = mpsc::channel::<IoOperation>(100);
5 let (cq_tx, mut cq_rx) = mpsc::channel::<IoOperation>(100);
6 loop {
7     let (stream, _) = listener.accept().await?;
8     tokio::spawn(async move {
9         loop {
10             let mut buf = Vec::new();
11             stream.read(&mut buf).await;
12             sq_tx.send(IoOperation::Read(stream, buf)).await;
13         }
14     });
15 }
16 tokio::spawn(async move {
17     while let Some(op) = sq_rx.recv().await {
18         tokio::spawn(async move {
19             let res = process(op.data).await;
20             cq_tx.send(IoOperation::Write(op.stream, res)).await;
21         });
22     }
23 });
24 tokio::spawn(async move {
25     while let Some(op) = cq_rx.recv().await {
26         op.stream.write_all(op.data).await;
27     }
28 });
29 Ok(())
30 }

```

13.4 24/25 Ticket Master

```

1 struct Seat {
2     mutex mtx; // For per-seat locking
3     atomic<bool> sold = false;
4     int reservedBy = -1;
5     steady_clock::time_point expiryTime = steady_clock::time_point::
        min();
6 };
7 class TicketSystem {
8     vector<Seat> seats;
9     thread expirationThread;
10    atomic<bool> running = true;
11
12    public:
13    TicketSystem(int numSeats) : seats(numSeats) {

```

```

14         expirationThread = thread(&TicketSystem::monitorExpirations,
15                                     this);
16     }
17     ~TicketSystem() {
18         running = false;
19         if (expirationThread.joinable()) {
20             expirationThread.join();
21         }
22     }
23     bool processPayment(int userID, int seatID) {
24         confirmPayment(userID);
25         return true;
26     }
27     bool reserveSeat(int userID, int seatID, int holdSeconds = 5) {
28         // (reservedBy + expiry for someone else holding seat)
29         if (seatID < 0 || seatID >= seats.size()) return false;
30         Seat& seat = seats[seatID];
31         unique_lock<mutex> lock(seat.mtx);
32         auto now = steady_clock::now();
33         if (seat.sold || (seat.expiryTime > now && seat.reservedBy
34             != userID)) {
35             return false; // Already sold or held by someone
36         }
37         seat.reservedBy = userID;
38         seat.expiryTime = now + seconds(holdSeconds);
39         return true;
40     }
41     bool confirmSeat(int seatID, int userID) {
42         if (seatID < 0 || seatID >= seats.size()) return false;
43         Seat& seat = seats[seatID];
44         unique_lock<mutex> lock(seat.mtx);
45         auto now = steady_clock::now();
46         if (seat.reservedBy == userID && seat.expiryTime > now) {
47             seat.sold = true;
48             seat.expiryTime = steady_clock::time_point::min();
49             return true;
50         } else {
51             refundUser(userID);
52             return false;
53         }
54     }
55     void monitorExpirations() {
56         while (true) {
57             this_thread::sleep_for(seconds(1));
58             auto now = steady_clock::now();
59             for (auto& seat : seats) {
60                 unique_lock<mutex> lock(seat.mtx);
61                 if (!seat.sold && seat.expiryTime < now) {
62                     seat.reservedBy = -1;

```

```

61         seat.expiryTime = steady_clock::time_point::min
62             ();
63     }
64 }
65 }
66 };
67
68 // atomics
69 struct Seat {
70     atomic<bool> available;
71     atomic<int> heldBy;
72     Seat() : available(true), heldBy(0) {}
73 };
74 class TicketSystem {
75     vector<Seat> seats;
76     // ...
77     bool reserveSeat(int userID, int seatID, int holdSeconds = 5) {
78         steady_clock::time_point newExpiry =
79             steady_clock::now() + seconds(holdSeconds);
80         if (seatID < 0 || seatID >= seats.size()) return false;
81         // Attempt to atomically reserve the seat
82         bool expected = true;
83         if (seats[seatID].available.compare_exchange_strong(expected
84             , false)) {
85             // Successfully changed availability to false
86             seats[seatID].heldBy.store(userID);
87             return true;
88         } else {
89             // Seat was not available
90             return false;
91         }
92     };

```

13.5 24/25 Load Balancer

```

1 type Request struct {
2     userID int
3     seatID int
4     responseCh chan bool
5 }
6 type Server struct {
7     id int
8     seatMin int
9     seatMax int
10    reqCh chan Request
11    ts *TicketServer

```

```

12 }
13 type LoadBalancer struct {
14     servers []*Server
15 }
16
17 func NewServer(id, seatMin, seatMax int) *Server {
18     s := &Server{
19         id: id,
20         seatMin: seatMin,
21         seatMax: seatMax,
22         reqCh: make(chan Request),
23         ts: NewTicketServer(seatMax),
24         // assume Server handles seatMin to seatMax
25     }
26     go func() {
27         for req := range s.reqCh {
28             success := s.ts.HandleClient(req.userID, req.seatID)
29             req.responseCh <- success
30         }
31     }()
32     return s
33 }
34
35 func NewLoadBalancer(numServers int, maxSeatID int) *LoadBalancer {
36     lb := &LoadBalancer{}
37     seatsPerServer := (maxSeatID + numServers - 1) / numServers
38     for i := 0; i < numServers; i++ {
39         seatMin := i * seatsPerServer
40         seatMax := seatMin + seatsPerServer - 1
41         if seatMax > maxSeatID {
42             seatMax = maxSeatID
43         }
44         lb.servers = append(lb.servers, NewServer(i, seatMin,
45             seatMax))
46     }
47     return lb
48 }
49
50 func (lb *LoadBalancer) Dispatch(userID int, seatID int, responseCh
51     chan bool) {
52     for _, s := range lb.servers {
53         if seatID >= s.seatMin && seatID <= s.seatMax {
54             s.reqCh <- Request{
55                 userID: userID,
56                 seatID: seatID,
57                 responseCh: responseCh,
58             }
59             return
60         }
61     }
62 }

```

```

59     }
60 }
61
62 func main() {
63     N := 3
64     maxSeatID := 9
65     lb := NewLoadBalancer(N, maxSeatID)
66     results := make([]chan bool, 20)
67     for i := 1; i <= 20; i++ {
68         seatID := rand.Intn(10)
69         results[i-1] = make(chan bool)
70         lb.Dispatch(i, seatID, results[i-1])
71     }
72     for i := 1; i <= 20; i++ {
73         confirmed := <-results[i-1]
74         fmt.Printf("User %d seat confirmation: %v\n", i, confirmed)
75     }
76     time.Sleep(5 * time.Second)
77 }

```